

Problem Set 1

Hoa T. Vu

Gradient Descent

Problem 1: Mean Squared Error. Given the curve $\hat{y} = 4x - 2$ and the data below, compute the MSE.

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

x	1	2	3	4
y	3	7	9	15

Problem 2: Compute the Gradient. Find ∇f for each function.

(a) $f(x, y) = x^2y + e^x$

(b) $f(x, y, z) = x \ln(y) + yz^2$

(c) $f(x, y) = \sin(xy) + x^3$

(d) $f(x, y) = \frac{x^2}{y+1}$

Problem 3: Derivative of Huber Loss.

The Huber loss with threshold $\delta > 0$ is defined as

$$\mathcal{L}_\delta(e) = \begin{cases} \frac{1}{2}e^2 & \text{if } |e| \leq \delta \\ \delta \left(|e| - \frac{\delta}{2} \right) & \text{if } |e| > \delta \end{cases}$$

where $e = y - \hat{y}$ is the residual.

(a) Write out $\frac{d\mathcal{L}_\delta}{de}$ as a piecewise function.

- (b) Show that $\mathcal{L}_\delta$ is differentiable at $e = \delta$ and at $e = -\delta$ (i.e., the left- and right-hand derivatives agree at each boundary point).
- (c) For a dataset of n points, the total loss is $\mathcal{L}(\hat{y}_1, \dots, \hat{y}_n) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}_\delta(y_i - \hat{y}_i)$. Suppose all predictions are a function of a single parameter θ , i.e. $\hat{y}_i = f(\theta)$. Use the chain rule to derive the update rule $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}}{\partial \theta}$, expressing the gradient in terms of e_i and $\frac{\partial \hat{y}_i}{\partial \theta}$.

Python Programming

Problem 4: Lists. Given the list below, write Python code for each task.

```
scores = [88, 72, 95, 61, 84, 90]
```

- Access the third score.
- Slice the last three scores.
- Append 77, then sort in descending order.
- Remove all scores below 80 using a list comprehension.

Problem 5: Sieve of Eratosthenes. Find all primes up to n by repeatedly crossing out multiples:

- Start with `is_prime = [True] * (n + 1)`; set `is_prime[0] = is_prime[1] = False`.
- For each p from 2 to $\lfloor \sqrt{n} \rfloor$: if `is_prime[p]` is still `True`, then p is prime: mark all its multiples starting from p^2 as composite: `is_prime[p*p]`, `is_prime[p*p + p]`, `is_prime[p*p + 2*p]`, ... We start at p^2 because all smaller multiples of p have already been crossed out by earlier primes.
- Collect all indices where `is_prime[i]` is `True`.

Tasks:

- Implement `sieve(n)` and verify it returns `[2, 3, 5, 7, 11, 13]` for $n = 13$.
- Use it to find the **1000th prime**.
- Count how many primes are below 10^6 . What is the running time of the sieve?